

神经网络与大语言模型的拓扑斯理论框架讲义

2026 年 6 月 26 日

摘要

本文是一份教科书式讲义，目标是解释近期一些“神经网络或大语言模型可以放入范畴论/拓扑斯框架”的工作。本文不把这些工作当成已经成熟的机器学习定理，而是先建立严格的范畴论语言，再说明这些语言在 AI 相关提案中如何被使用。讲义从范畴、函子、自然变换、极限、伴随、预层、Yoneda 引理、层、site、Grothendieck 拓扑、Grothendieck 拓扑斯、内部逻辑和 stack 讲起，然后把 Lafforgue、Belfiore–Bennequin、Transformer topos、Mahadevan 和 Jia–Peng–Yang–Chen 综述作为案例来读。重点是区分：哪些是经典数学，哪些是结构表示，哪些是语义解释，哪些仍只是研究纲领。

1 中心问题

神经网络通常被写成带参数的映射

$$N_\theta : X \longrightarrow Y$$

或者若干层的复合

$$N_\theta = f_{L,\theta_L} \circ f_{L-1,\theta_{L-1}} \circ \cdots \circ f_{1,\theta_1}.$$

这种写法当然正确：给定参数 θ 后，网络确实定义了一个从输入空间到输出空间的函数。但是它主要记录的是“整体输入–输出行为”。如果只看这个函数，许多实现层面的结构会被压缩掉。

例如，两个网络可能作为函数 $X \rightarrow Y$ 完全相同，却有不同的中间层表示；反过来，两个中间表示也可能不是字面相等，而只是差一个可逆坐标变换。若在第 l 层把特征坐标变换为

$$h_l : V_l \longrightarrow V_l,$$

并相应地把相邻层的权重作共轭式调整，那么整体复合 N_θ 可能不变，但中间表示的坐标已经改变。因此，单独写

$$N_\theta : X \rightarrow Y$$

不会告诉我们哪些表示应当视为“同一个表示的不同坐标描述”。

同样，层复合公式虽然显示了计算顺序，却没有自动说明“局部”是什么意思。局部可以指图像 patch、token 子窗口、网络层、attention head、任务上下文，或者某个语义片段。要讨论“局部

信息能否粘成全局信息”，必须先指定局部片段的范畴以及哪些片段构成覆盖。函数记号本身不包含这些选择。

所以这里的意思不是说普通写法不对，而是说它太外部化了：它把网络看成一个整体函数，却没有显式记录局部数据、限制映射、坐标等价、上下文依赖和局部到全局的粘合规则。层和拓扑斯语言正是试图把这些额外结构放到数学对象本身里。

拓扑斯式方法从另一端出发。它希望找到一个上下文范畴 $\mathcal{C}$ 、一个覆盖结构 J ，以及层范畴

$$\mathbf{Sh}(\mathcal{C}, J).$$

这个层范畴被解释为一个由上下文相关对象组成的语义宇宙。

原则 1.1 (负责任的阅读方式). 当一篇文章说神经网络、*Transformer* 或 *LLM* 有拓扑斯结构时，应当先问四个问题：

1. 范畴 $\mathcal{C}$ 是什么？
2. 态射是什么？
3. 拓扑 J 是什么，即哪些族算覆盖？
4. 具体使用的是层、拓扑斯、*stack*，还是某个泛构造？

如果这些问题没有答案，“*topos*”更像比喻，而不是定理。

2 范畴、函子与自然变换

定义 2.1 (范畴). 范畴 $\mathcal{C}$ 由对象、态射集合 $\mathrm{Hom}_{\mathcal{C}}(A, B)$ 、恒等态射 $\mathrm{id}_A : A \rightarrow A$ 和复合运算组成，并满足结合律和单位律。

这个定义的重点是关系，而不是对象内部的材料。范畴论关心对象如何互相映射，以及映射如何复合。

定义 2.2 (函子). 函子 $F : \mathcal{C} \rightarrow \mathcal{D}$ 把对象送到对象，把态射送到态射，并保持恒等与复合：

$$F(\mathrm{id}_A) = \mathrm{id}_{F(A)}, \quad F(g \circ f) = F(g) \circ F(f).$$

例子 2.3 (遗忘函子). 遗忘函子

$$U : \mathbf{Grp} \longrightarrow \mathbf{Set}$$

把群 $G = (|G|, \cdot, e, (-)^{-1})$ 送到底层集合 $|G|$ ，把群同态送成同一个底层函数。它不是取生成元集合，而是保留所有元素并忘掉群运算。

定义 2.4 (自然变换). 若 $F, G : \mathcal{C} \rightarrow \mathcal{D}$ 是函子，自然变换 $\eta : F \Rightarrow G$ 给每个对象 A 一个态射

$$\eta_A : F(A) \rightarrow G(A),$$

并要求对每个 $f : A \rightarrow B$ 有

$$G(f) \circ \eta_A = \eta_B \circ F(f).$$

自然变换是函子之间的态射。之后预层本身就是函子，所以预层之间的态射就是自然变换。

3 极限：相容数据的对象

定义 3.1 (拉回). 给定 $f : A \rightarrow C$ 和 $g : B \rightarrow C$, 拉回是对象 $A \times_C B$, 带投影

$$p_1 : A \times_C B \rightarrow A, \quad p_2 : A \times_C B \rightarrow B,$$

使得

$$f \circ p_1 = g \circ p_2,$$

并且对所有满足同样相容条件的对象具有泛性质。

例子 3.2. 在 **Set** 中,

$$A \times_C B = \{(a, b) \in A \times B \mid f(a) = g(b)\}.$$

因此拉回就是满足共同约束的配对集合。

说明 3.3. 若图像表示 I 和文本表示 T 都映到共同语义空间 S , 则 $I \times_S T$ 表示语义一致的图文配对。这还不是拓扑斯理论, 但它展示了泛构造如何表达模型结构。

有限极限, 包括终对象、积、等化子和拉回, 是表达合取、等式、替换和上下文扩张的基础。因此 elementary topos 的定义要求有限极限。

4 伴随

定义 4.1 (伴随). 设 $L : \mathcal{C} \rightarrow \mathcal{D}$ 和 $R : \mathcal{D} \rightarrow \mathcal{C}$ 是函子。伴随 $L \dashv R$ 是自然双射

$$\mathrm{Hom}_{\mathcal{D}}(LC, D) \cong \mathrm{Hom}_{\mathcal{C}}(C, RD).$$

例子 4.2 (自由群与遗忘函子). 自由群函子 $F : \mathbf{Set} \rightarrow \mathbf{Grp}$ 左伴随于遗忘函子 $U : \mathbf{Grp} \rightarrow \mathbf{Set}$, 因为

$$\mathrm{Hom}_{\mathbf{Grp}}(F(X), G) \cong \mathrm{Hom}_{\mathbf{Set}}(X, U(G)).$$

这不是说二者互逆, 而是说从自由群 $F(X)$ 到 G 的群同态等价于从生成集合 X 到 G 底层集合的普通函数。

伴随是拓扑斯理论中的核心结构。层化是层范畴嵌入预层范畴的左伴随; 几何态射是一对伴随函子; 内部逻辑中的量词也可以用伴随表达。

5 预层与 Yoneda 引理

定义 5.1 (预层). 给定范畴 $\mathcal{C}$, 其上的预层是反变函子

$$F : \mathcal{C}^{op} \longrightarrow \mathbf{Set}.$$

预层范畴记为

$$\mathbf{PSh}(\mathcal{C}) = [\mathcal{C}^{op}, \mathbf{Set}].$$

反变性表达限制。若 $U \rightarrow V$ 表示 U 是 V 中较小的上下文，则 V 上的数据可以限制到 U 上：

$$F(V) \longrightarrow F(U).$$

定义 5.2 (可表预层). 对 $A \in \mathcal{C}$ ，可表预层 yA 定义为

$$yA = \text{Hom}_{\mathcal{C}}(-, A).$$

也就是说， $yA(X) = \text{Hom}_{\mathcal{C}}(X, A)$ 。

引理 5.3 (Yoneda 引理). 对任意预层 $F : \mathcal{C}^{op} \rightarrow \mathbf{Set}$ 和对象 $A \in \mathcal{C}$ ，有自然双射

$$\text{Nat}(yA, F) \cong F(A).$$

证明. 设 $\eta : yA \Rightarrow F$ 是自然变换。定义

$$a = \eta_A(\text{id}_A) \in F(A).$$

对任意 $f : X \rightarrow A$ ，由自然性得到

$$\eta_X(yA(f)(\text{id}_A)) = F(f)(\eta_A(\text{id}_A)).$$

因为 $yA(f)$ 是用 f 作预复合，

$$yA(f)(\text{id}_A) = \text{id}_A \circ f = f.$$

所以

$$\eta_X(f) = F(f)(a).$$

因此整个自然变换由 $a \in F(A)$ 决定。

反过来，给定 $a \in F(A)$ ，定义

$$\eta_X(f) = F(f)(a).$$

函子性保证这些映射满足自然性。两种构造互逆。 $\square$

说明 5.4. Yoneda 引理说明， $F(A)$ 中的元素等价于一种自然规则：每当有广义元素 $X \rightarrow A$ ，就产生对应的 F -数据。

6 层、Site 与层化

定义 6.1 (Grothendieck 拓扑与 site). Grothendieck 拓扑 J 给每个对象 U 指定哪些族

$$\{U_i \rightarrow U\}_{i \in I}$$

算作覆盖。带有这种覆盖结构的范畴 $(\mathcal{C}, J)$ 称为 *site*。

定义 6.2 (层). 预层 $F : \mathcal{C}^{op} \rightarrow \mathbf{Set}$ 是层，如果每个覆盖上的相容局部截面都能唯一粘成全局截面。

在拓扑空间的情形中，如果 $U = \bigcup_i U_i$ ，且 $s_i \in F(U_i)$ 满足

$$s_i|_{U_i \cap U_j} = s_j|_{U_i \cap U_j},$$

则存在唯一 $s \in F(U)$ ，使得

$$s|_{U_i} = s_i.$$

例子 6.3 (有界连续函数). 在 $X = \mathbb{R}$ 上令

$$F(U) = \{f : U \rightarrow \mathbb{R} \mid f \text{ 连续且有界}\}.$$

这是预层但不是层。函数 $f_n(x) = x$ 在 $(-n, n)$ 上有界连续，并且彼此相容，但它们粘成的 $\mathbb{R}$ 上函数 $f(x) = x$ 不有界。

定义 6.4 (层化). 设 $i : \mathbf{Sh}(\mathcal{C}, J) \hookrightarrow \mathbf{PSh}(\mathcal{C})$ 是包含函子。层化是左伴随

$$a : \mathbf{PSh}(\mathcal{C}) \rightarrow \mathbf{Sh}(\mathcal{C}, J), \quad a \dashv i.$$

层化修补两个问题。第一，局部相同的截面应被识别。第二，相容的局部截面应被加入为全局粘合。在有界连续函数例子中，层化得到普通连续函数层，因为连续实值函数总是局部有界。

7 Grothendieck 拓扑斯与内部逻辑

定义 7.1 (Grothendieck 拓扑斯). *Grothendieck* 拓扑斯是等价于某个 *site* 上层范畴的范畴：

$$\mathcal{E} \simeq \mathbf{Sh}(\mathcal{C}, J).$$

Grothendieck 拓扑斯有三种读法：广义空间、上下文相关集合的宇宙，以及具有内部逻辑的语义宇宙。

定义 7.2 (子对象分类器). 在 *elementary topos* 中，子对象分类器是对象 Ω ，使得子对象 $S \hookrightarrow X$ 由态射

$$\chi_S : X \rightarrow \Omega$$

分类。在 **Set** 中， $\Omega = \{0, 1\}$ 。

内部逻辑把谓词解释为子对象，把真值解释为到 Ω 的态射。对 AI 而言，只有在明确了哪些对象是语义对象、哪些子对象是谓词、哪些态射表示证据或限制之后，内部逻辑才真正有内容。

7.1 为什么有限极限、指数对象和真值对象会一起出现

elementary topos 的定义初看很奇怪：有限极限、指数对象、子对象分类器。它们放在一起的原因是，这三类结构分别对应普通集合论推理里的基本语法。

有限极限提供“上下文”的语法。积 $X \times Y$ 表示同时拥有一个 X -变量和一个 Y -变量的上下文。拉回表示代入：若某个谓词或构造定义在 Y 上，而我们有态射 $X \rightarrow Y$ ，那么拉回把这个构造搬到 X -上下文。等化子表示方程。

指数对象提供函数类型。指数对象 Y^X 由自然双射

$$\mathrm{Hom}_{\mathcal{E}}(Z \times X, Y) \cong \mathrm{Hom}_{\mathcal{E}}(Z, Y^X)$$

刻画。它说，一个由 Z 参数化的 $X \rightarrow Y$ 映射族，等价于一个到函数对象 Y^X 的映射 $Z \rightarrow Y^X$ 。用机器学习语言说，这对应于把模型族看成外部的一堆函数，还是看成范畴内部的一个函数对象。

子对象分类器提供真值。若 $S \hookrightarrow X$ 是 X 上的谓词，那么特征态射

$$\chi_S : X \rightarrow \Omega$$

就是这个谓词的内部真值函数。在 **Set** 中，真值只有 0, 1。在层拓扑斯中，一个命题的真值可以是局部的：它可能在某些上下文片段上为真，在另一些片段上不成立。这就是“内部逻辑”的来源。它不是说拓扑斯自己就是 AI 推理器，而是说这个范畴本身支持一种逻辑语义。

7.2 量词作为伴随

设 $p : X \rightarrow Y$ 是拓扑斯中的态射。沿 p 拉回会把 Y 上的谓词变成 X 上的谓词：

$$p^* : \mathrm{Sub}(Y) \longrightarrow \mathrm{Sub}(X).$$

在通常的逻辑情形中，这个函子有左右伴随

$$\exists_p \dashv p^* \dashv \forall_p.$$

左伴随 $\exists_p$ 表示沿 p 的纤维做存在量化，右伴随 $\forall_p$ 表示沿纤维做全称量化。于是我们之前说“伴随是量词的范畴形式”不是比喻，而是一个可检查的结构事实。

因此，若一篇 AI 文章声称某个模型有内部逻辑，就必须说明：谓词是什么，上下文态射是什么，相关拉回函子是否真的有这些伴随。否则“存在”“所有”“真值”这些词仍停留在直觉层。

8 Stack

层粘合集合值局部数据。Stack 粘合范畴值局部数据，不仅粘合对象，还粘合对象之间的同构。若神经表示不是严格相等，而是在对称、重参数化、基变换或规范变换意义下等价，stack 语言就比层更合适。

例如，两个 attention head 或两个隐藏特征空间可能在坐标变换后实现同一功能角色。若要求字面相等，层语言可能太严格；stack 语言则能记录等价及其相容性。

9 一个小模型：上下文索引的预测

在读关于 LLM 的宏大说法之前，先看一个可以检查的玩具模型。令 $\mathcal{W}$ 是某个固定文档中有限 token window 按包含关系组成的偏序范畴。也就是说，当且仅当 $U \subseteq V$ 时有唯一态射 $U \rightarrow V$ 。

定义预层 $P : \mathcal{W}^{op} \rightarrow \mathbf{Set}$ ：令 $P(U)$ 是窗口 U 上可得到的一组 next-token 分布或预测状态。若 $U \subseteq V$ ，则态射 $U \rightarrow V$ 在预层中给出限制映射

$$P(V) \longrightarrow P(U).$$

它的含义是：在较大窗口 V 中得到的预测，可以被限制到较小窗口 U 中可见的信息。

这个例子解释了为什么预层是反变的：更多上下文可以限制到更少上下文，而不是反过来。它也解释了预层和层的差别。设大窗口 V 被重叠子窗口 U_i 覆盖。即使有一族局部预测 $p_i \in P(U_i)$ ，并且它们在重叠处相容，也未必存在一个全局预测 $p \in P(V)$ 限制后得到所有 p_i 。若这种情况发生， P 对这个覆盖就不是层。层化会形式地加入缺失的粘合，并识别局部不可区分的预测。

但这不是思维链的完整数学模型。思维链中常有临时判断、矛盾、修正和反思。层条件本质上是相容性原则，它本身不擅长表达“通过矛盾和修正逐步逼近答案”。若要建模思维链，更需要一个态射能记录修正、反驳、细化、证明搜索的基范畴，而不只是子窗口限制。这正说明：选择 $\mathcal{C}$ 、 J 和态射含义不是技术细节，而是决定形式系统能表达什么。

10 怎样把 AI 主张变成定理

很多文章会出现类似句子：

“某类神经网络结构生活在一个拓扑斯中。”

要把这句话变成定理，至少需要以下数据。

1. 定义神经网络对应的上下文范畴 $\mathcal{C}_N$ 。对象可以是层、特征空间、架构状态、token window、prompt、任务或语义情境；态射必须是真正保持结构的映射，而不是类比。
2. 定义 Grothendieck 拓扑 J_N 。这一步说明哪些上下文族算覆盖，也就是你希望采用哪种局部到全局原则。
3. 定义表示神经数据的预层或层。例如，一个预层可以把每个上下文送到局部状态、局部参数、局部预测或局部语义断言的集合。
4. 证明层条件，或者说明为什么必须层化。
5. 说明保持的性质。一个有用的定理不应只说“存在表示”，还应说明这个表示保持复合、对称性、不变性、语义一致性、表达能力或某个明确命名的架构性质。

原则 10.1 (没有免费的拓扑斯)。不能因为一个系统复杂就说它自然形成拓扑斯。要得到拓扑斯，要么构造 *site* 并取其层范畴，要么对某个指定范畴证明 *elementary topos* 的公理。

11 如何阅读 Topos-Based AI 主张

原则 11.1 (四层分类). 阅读相关文献时, 应把每个主张放入四层之一:

1. 经典拓扑斯理论: 已经成熟的定义和定理;
2. 结构表示: 把神经对象编码进范畴、拓扑斯或 *stack*;
3. 语义解释: 把范畴结构解释为意义、上下文、真值或解释;
4. 算法后果: 真正的训练、推理、鲁棒性或泛化结论。

目前大部分工作在前两层较强, 在第三层有启发性, 在第四层仍较早。

12 Lafforgue 的纲领: 拓扑斯作为语义补全

Lafforgue 的想法适理解解为语义补全纲领。从一个有意义对象的范畴 $\mathcal{C}$ 出发, 例如几何、符号或知识结构。Yoneda 嵌入给出

$$y : \mathcal{C} \hookrightarrow \mathbf{PSh}(\mathcal{C}).$$

再选择 Grothendieck 拓扑 J , 得到层范畴

$$\mathbf{Sh}(\mathcal{C}, J).$$

数学内容是标准拓扑斯理论: 预层与层范畴提供了包含原始对象的更丰富宇宙。AI 解释是, 数值学习应当连接回语义或几何来源范畴。这是研究纲领; 只有当 $\mathcal{C}$ 、 J 和语义映射被明确给出时, 才成为定理。

对学习者而言, 这个纲领最有价值的地方不是“AI 就是拓扑斯”, 而是它把语义建模分成三层:

$$\text{原始数据} \longrightarrow \text{结构化对象} \longrightarrow \text{局部相容解释组成的层}.$$

Yoneda 嵌入说明, 小的来源范畴可以嵌入到一个预层宇宙中。Grothendieck 拓扑 J 则说明哪些局部片段可以作为全局解释的证据。因此真正的数学问题是选择 J , 使层条件符合目标语义, 而不是机械地要求所有东西都能粘合。

原则 12.1 (Lafforgue 纲领真正使用的定理). 已经成熟的定理是: 给定 *site*, 可以构造预层和层拓扑斯, 并且它们具有丰富的泛性质和内部逻辑。AI 相关部分是: 如何选择 *site*, 使其捕捉学习中真正重要的语义结构。

13 Belfiore–Bennequin: DNN、拓扑斯与 Stack

Belfiore 和 Bennequin 提出, 深度神经网络可以表示为 canonical Grothendieck topoi 中的对象, 并且网络中的不变性结构可以用 *stacks* 描述。

数学读法如下。神经网络不应只被看成映射 $N_\theta : X \rightarrow Y$ 。它还有层、状态、参数、对称性、特征空间和变换。于是可以尝试构造一个网络上下文范畴，并在其上建立层或 stack。拓扑斯提供全局语义宇宙；stack 用于编码局部等价和不变量。

这不是关于泛化能力的证明，而是结构表示提案。若要在具体情况下变得完全严格，必须说明：

1. 网络上下文的基范畴是什么；
2. 覆盖族是什么；
3. 哪些对象或层表示层、状态或参数；
4. stack 需要粘合哪些等价。

stack 出现的原因值得细看。设局部特征表示只在可逆坐标变换下确定。若我们用局部区域覆盖一个网络，得到局部表示 E_i ，那么在重叠区域上 E_i 与 E_j 不一定相等，而可能只是同构。集合值层要求限制后相等；stack 则允许在重叠处给出同构，并要求这些同构在三重重叠上满足 cocycle 条件。这正是“相同表示”应理解为“在对称变换下相同”时需要的语言。

因此，Belfiore–Bennequin 最强的数学读法应是：

深度网络不应只作为函数研究，而应作为带有局部对称性的结构对象研究；这些
局部对称性的下降数据可以由拓扑斯和 stack 组织。

这是严肃的结构表示提案，但它本身不是优化算法，也不是“训练后的 DNN 已经学到了正确语义 stack”的证明。那些更强结论需要额外假设和证明。

14 Transformer Topos 提案

The Topos of Transformer Networks 在 arXiv 上已标注为 withdrawn，因此不能作为稳定定理引用。它有启发性的想法是：由于 attention 是上下文依赖的，并具有某种高阶计算特征，transformer 可能需要比普通分段线性网络范畴更丰富的范畴环境。

负责任的数学问题是：

attention 型上下文计算是否迫使我们从较弱的神经映射范畴走向一个具有指数对象、子对象和内部逻辑的更丰富补全？

要证明这一点，需要精确定义 pre-transformer 网络范畴、补全构造，并证明 transformer 操作存在于补全中但不在较小范畴中。

“topos completion”这个词也要谨慎理解。一个补全定理通常有如下形式：从缺少某些运算的范畴 $\mathcal{A}$ 出发，构造一个范畴 $C(\mathcal{A})$ 和函子

$$\iota : \mathcal{A} \rightarrow C(\mathcal{A}),$$

使 $C(\mathcal{A})$ 具有目标结构，并且对所有具有这些结构的范畴满足某个泛性质。因此，补全不是“变得更丰富”的比喻，而是一个由泛性质刻画的结构。

对 Transformer 而言，合理的数学希望是：attention 的上下文计算可能需要类似内部函数空间、依赖上下文求值或子对象逻辑的结构。要使其严格化，必须指出哪个 transformer 操作不能在较

小范畴中表示，并证明补全恰好加入了所需结构。由于该论文已撤回，本讲义把它当作未来形式化的提示，而不是可靠定理。

15 Mahadevan 关于生成式 AI 与 LLM 的提案

Mahadevan 的想法是用拓扑斯中的泛构造指导生成式 AI 和 LLM。关键主张是，适当定义的 LLM-like 系统范畴可能具有极限、余极限、指数对象和类似子对象分类器的结构。

这在数学上要求很高。若要证明某个 LLM 范畴 $\mathcal{L}$ 是拓扑斯，至少要证明：

1. $\mathcal{L}$ 有有限极限；
2. $\mathcal{L}$ 有指数对象；
3. $\mathcal{L}$ 有子对象分类器；
4. $\mathcal{L}$ 的态射定义使这些结构有实际意义。

否则，“LLM 构成拓扑斯”更像设计原则，而不是定理。

作为设计原则，它仍然有用。Pullback 可表示语义对齐，pushout 可表示模块融合，equalizer 可表示模型输出一致部分，coequalizer 可表示按行为等价取商。

一个更保守的使用方式是：让 $\mathcal{L}$ 的对象表示模型接口，而不是完整的物理训练过程；让态射表示保持接口的变换。这样，有限极限和余极限有时可以在接口层面构造，即使所有训练权重及任意微调操作未必形成干净的范畴。这个区别很重要：API、prompt、行为或语义规格的范畴可能比较可控；“所有训练好的 LLM 加上所有微调操作”的范畴则困难得多。

因此 Mahadevan 的提案可以分两层读。第一层是范畴层：哪些泛构造有助于组合生成式系统？第二层是拓扑斯层：这些构造是否真的扩展到一个具有有限极限、指数对象和子对象分类器的语义宇宙？第一层较容易形式化；第二层必须完成上面列出的 topos 检查。

16 Jia-Peng-Yang-Chen 综述

Jia、Peng、Yang、Chen 的 2025 年综述不是新定理，而是文献地图。它的价值在于把较成熟的应用范畴论机器学习与更探索性的拓扑斯路线分开。基于梯度的范畴学习、Markov categories、lenses、equivariant learning 等更接近成熟框架；Grothendieck 拓扑斯、stack 和内部逻辑则仍是更探索的层次。

这篇综述也提醒我们不要滥用词汇。并非所有用于机器学习的范畴论方法都是拓扑斯理论。过程的 monoidal category、概率映射的 Markov category、双向更新的 lens category、上下文数据的 sheaf topos 解决的问题不同。它们可以相互作用，但不能被压缩成一个口号。

17 什么已经证明，什么还没有

这些工作中最严格的数学内容主要是范畴论语言本身：预层、层、层化、Grothendieck 拓扑斯、stack 和内部逻辑。这些是成熟概念。

AI 相关主张多数是结构性的。它们说某类神经架构可以编码进某个范畴环境，或某种拓扑斯补充提供合适的语义宇宙。这样的主张有价值，前提是编码确实保留了架构的重要性质，而不是任意编码。

未来最强的结果应当是如下形式：

给定一个精确定义的神经系统类 $\mathcal{N}$ ，存在 canonical site $(\mathcal{C}_{\mathcal{N}}, J_{\mathcal{N}})$ ，使得 $\mathcal{N}$ 的语义由 $\mathbf{Sh}(\mathcal{C}_{\mathcal{N}}, J_{\mathcal{N}})$ 中指定的对象、层或 stack 表示，并且该表示保持某个明确命名的架构性质。

18 参考文献

- Jean-Claude Belfiore and Daniel Bennequin, [Topos and Stacks of Deep Neural Networks](#).
- Laurent Lafforgue, [Some Possible Roles for AI of Grothendieck Topos Theory](#).
- Mattia Jacopo Villani and Peter McBurney, [The Topos of Transformer Networks](#). 该文 arXiv 页面标注为 withdrawn and requiring major revision。
- Sridhar Mahadevan, [Topos Theory for Generative AI and LLMs](#).
- Yiyang Jia, Guohong Peng, Zheng Yang, and Tianhao Chen, [Category-Theoretical and Topos-Theoretical Frameworks in Machine Learning: A Survey](#).
- Saunders Mac Lane and Ieke Moerdijk, *Sheaves in Geometry and Logic*.
- Steve Awodey, *Category Theory*.